

robbit System Manual

Kise Laboratory
Created: September 2025

- This manual describes the hardware, software, and gateway of robbit. Here, “gateway” refers to the circuitry implemented in the FPGA.
- This manual helps readers understand the robbit system and improve its software and gateway.

robbit: Hardware

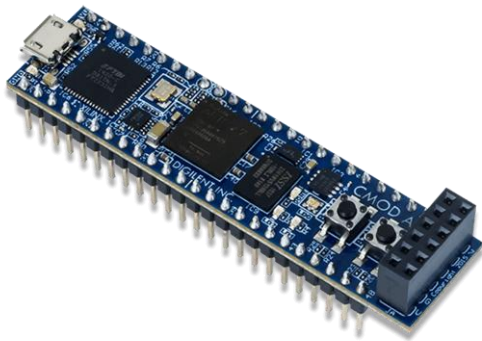
■ FPGA board: Cmod A7-35T

- Dimensions: 69.9 mm × 18.8 mm
- Weight: 10.7 g

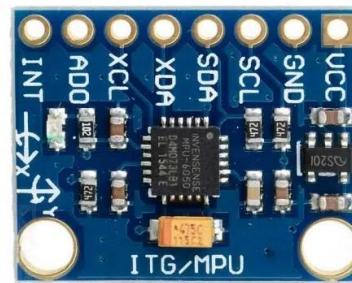
■ Sensor: MPU-6050 (IMU)

- Accelerometer and angular-velocity (gyroscope) sensor

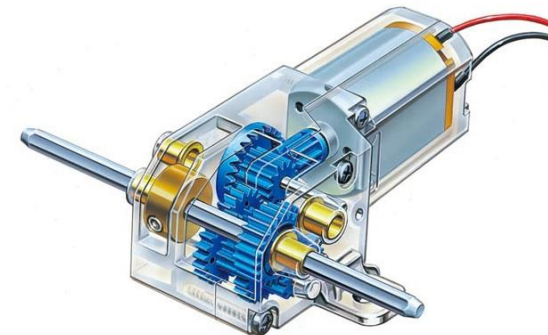
■ Geared motor: Mini Motor Standard Gearbox (TAMIYA)



Cmod A7-35T



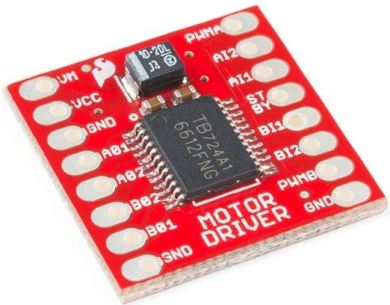
MPU-6050



Mini Motor Standard Gearbox (TAMIYA)

robbit: Hardware

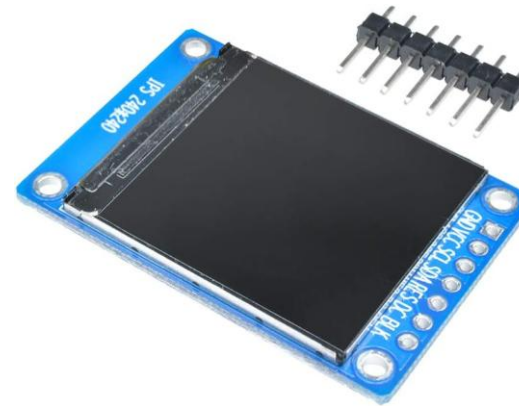
- Motor driver: TB6612FNG motor-driver module
- Tires: Slim Tire Set (55 mm diameter), 70193
- Display: 240 × 240 mini display with ST7789
- Battery: EEMB 653042 (3.7 V)



TB6612FNG
Motor Driver



Slim Tire Set

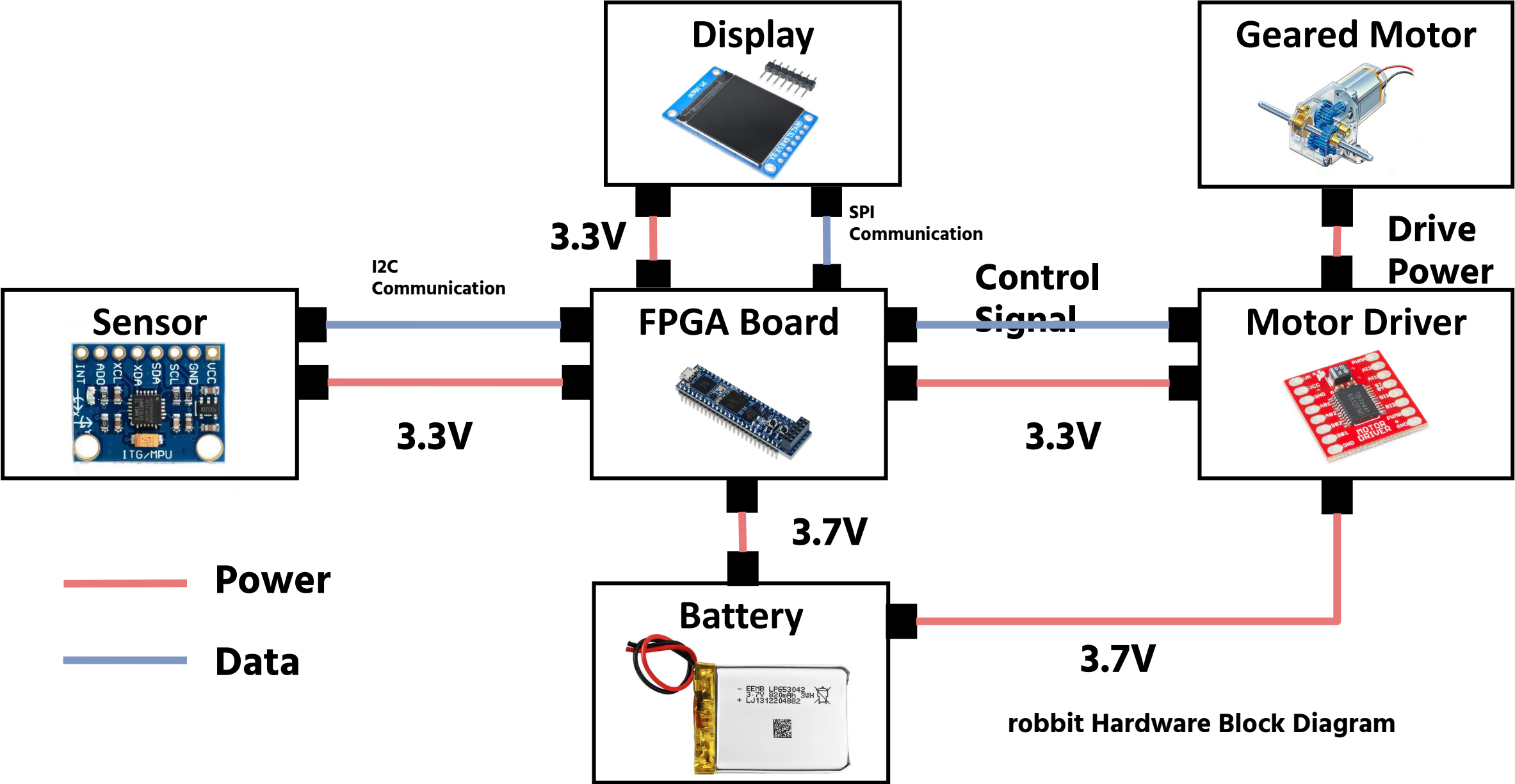


ST7789
Mini Display



EEMB 653042
Rechargeable
Battery

robbit: Hardware

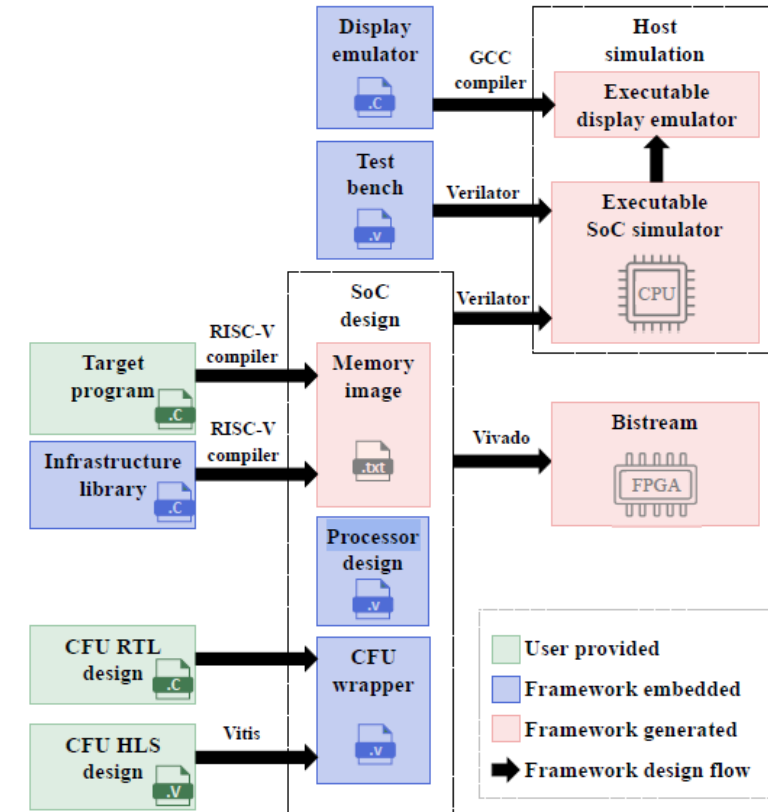


Gateway: Overview

■ Uses CFU-Proving-Ground

- Supports high-speed control up to 175 MHz
- Supports a mini display
- Enables flexible Memory-Mapped I/O (MMIO) design

Adds motor-driver and IMU modules to CFU-Proving-Ground



[1] Aoba Fuzino, Kenji Kise. “CFU Proving Ground,” <https://github.com/archlab-sciencetokyo/CFU-Proving-Ground>. Accessed: 2025-09-15.

Gateway: Overview

The Verilog files perform the following functions:

- main.v

SPI communication with the display

- mpu.v

I2C data transfer with the IMU

- proc.v

Implements a 32-bit RISC-V processor

- robbit.v

Motor-driver control module (tb6612fng) for PWM control

MMIO module: Reads and writes the memory region added for robbit

mpu.v and robbit.v are modules added to CFU Proving Ground.

Gateway: I2C Communication Module

- The FPGA communicates with the MPU-6050 over I2C.
- The mpu6050 and i2c_master modules in mpu.v handle this communication.

mpu6050 module: Controls the timing of I2C data transfers

i2c_master module: Transfers data between the IMU and FPGA

- I2C uses two signal lines, SCL and SDA.

SCL carries the clock, while SDA carries data.

- Data is transferred one bit at a time on SDA in synchronization with SCL.



Gateway: tb6612fng Module

- robbit.v contains the tb6612fng module that controls the motor driver.
- It sends PWM control signals to the motor driver.
- PWM Control
 - Motor output is controlled by the signal's ON/OFF ratio (duty cycle).

Higher ON ratio = faster motor rotation

Lower ON ratio = slower motor rotation

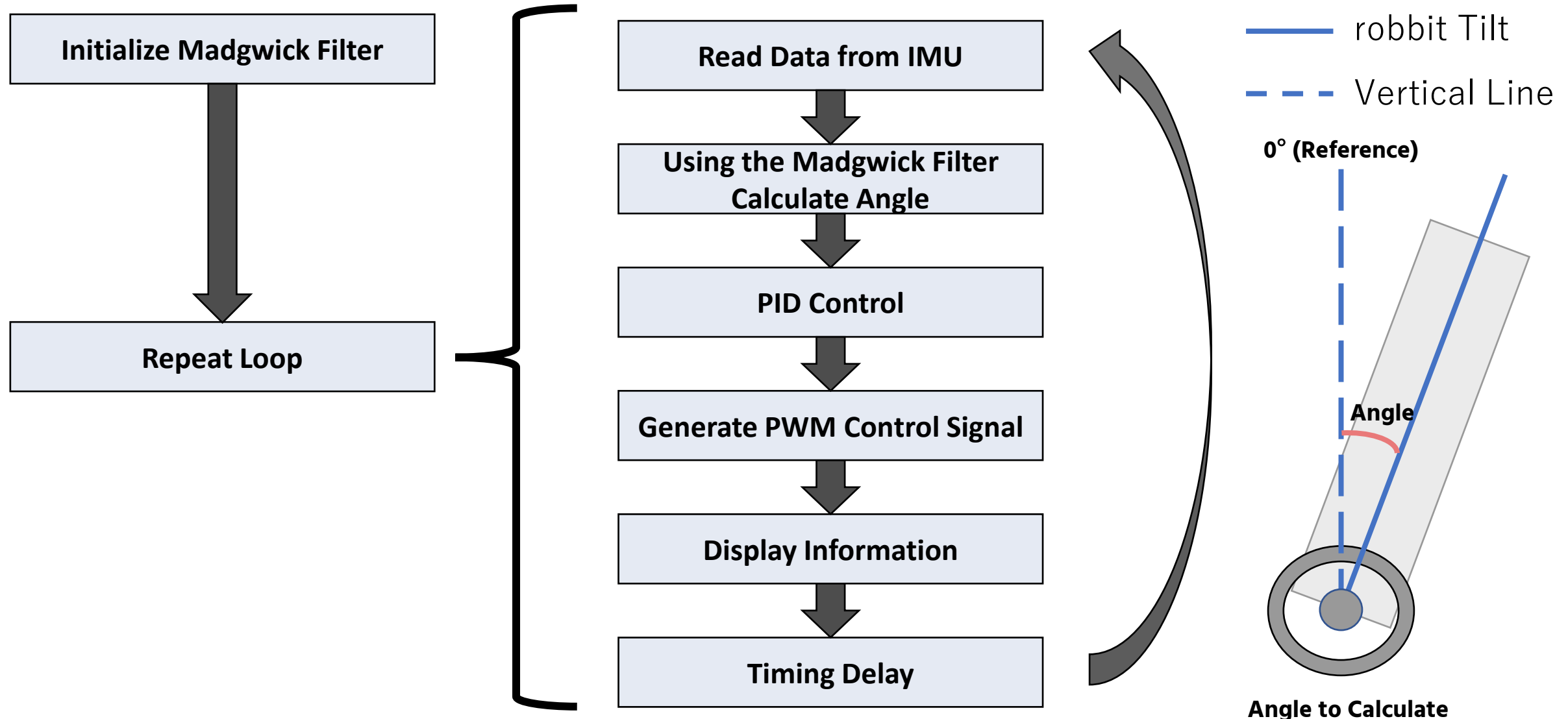


Faster Motor Rotation



Slower Motor Rotation

robbit: Software



Software: Reading Data from the IMU

- Read three-axis acceleration and angular velocity from the IMU.
- The gateway writes acceleration and angular-velocity data through Memory-Mapped I/O (MMIO), and the software reads the corresponding addresses.

```
//get acceralation and angular velocity
int16_t ax, ay, az, gx, gy, gz;
unsigned int data;
data = *(MPU_ADDR_ayax);
ax = data & 0xffff;      Read X- and Y-axis acceleration
ay = data >> 16;

data = *(MPU_ADDR_gxaz);
az = data & 0xffff;      Read Z-axis acceleration and X-axis angular velocity
gx = data >> 16;

data = *(MPU_ADDR_gzgy);
gy = data & 0xffff;      Read Y- and Z-axis angular velocity
gz = data >> 16;

//calculation of roll
MadgwickFilter.updateIMU(-gz / 131.0, gy / 131.0, gx / 131.0,
                        -az / 16384.0, ay / 16384.0, ax / 16384.0);
roll = MadgwickFilter.getRoll();
```

Software: Madgwick Filter

- The angle is calculated from three-axis acceleration and angular velocity using a Madgwick filter.
- The Madgwick-filter implementation is stored as a library in the Madgwick directory.

■ PID Control

- The control output (angle) is determined by combining the proportional, integral, and derivative terms.
- The influence of each term is determined by its gain (P_GAIN, I_GAIN, D_GAIN).

$$\text{Power} = \text{P_GAIN} \times \text{P} + \text{I_GAIN} \times \text{I} + \text{D_GAIN} \times \text{D}$$

PID Control: Proportional Term (P)

- P term = error between the current angle and target angle.
- Larger P_GAIN makes the system more sensitive to deviations.

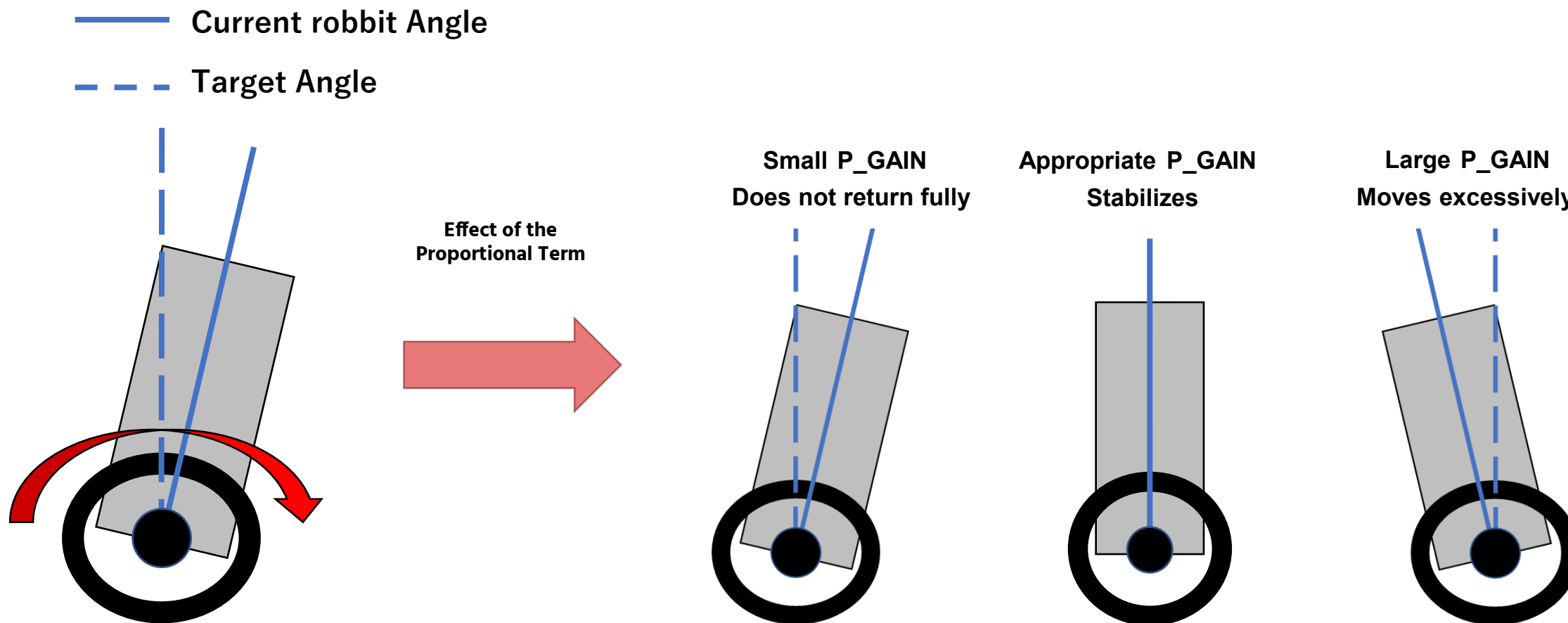


Illustration of the Proportional Term's Effect

PID Control: Integral Term (I)

- I term integrates the error between the current angle and target angle over time.
- Larger I_GAIN reduces deviation, but too much accumulation worsens response and stability.

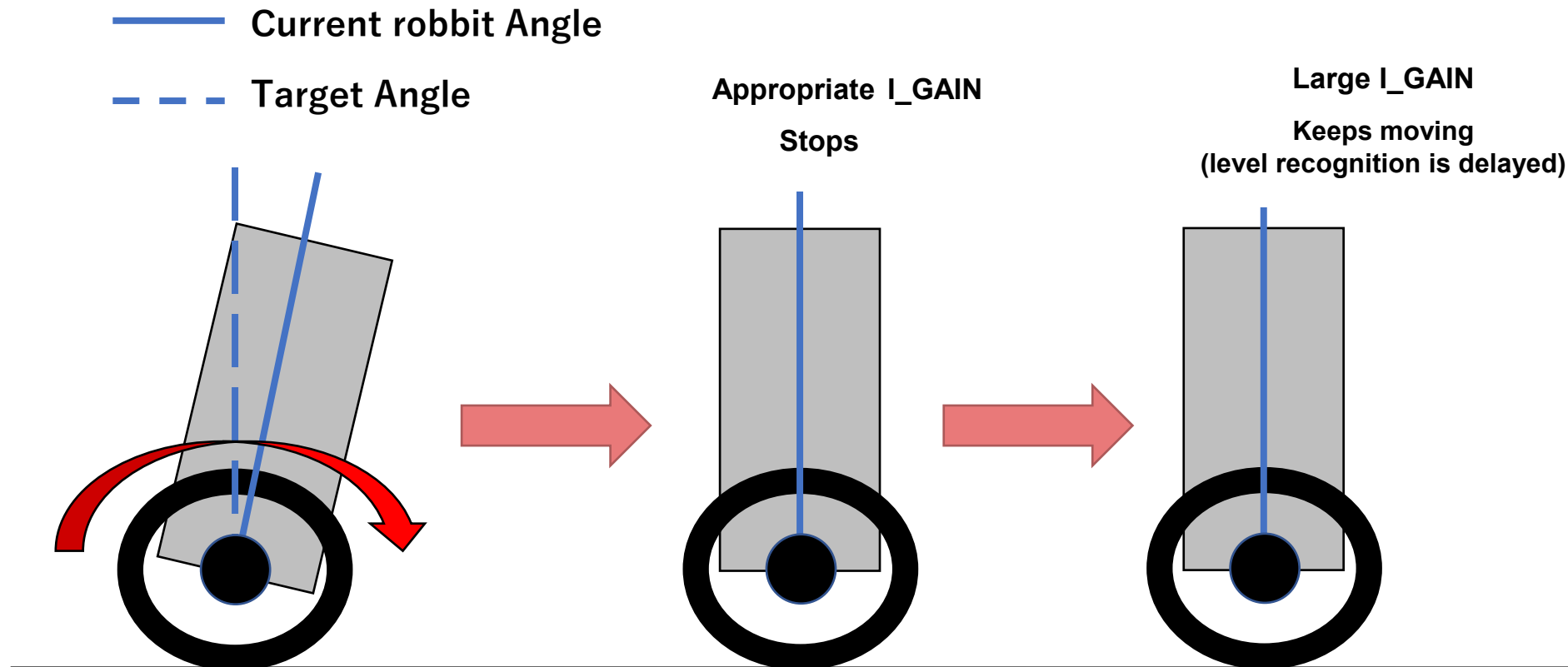


Illustration of the Integral Term's Effect

PID Control: Anti-Windup

- Anti-windup is used to suppress accumulation of the integral term described on the previous page.
- One method is to set upper and lower limits for the calculated integral term and update it only within that range.

$$I = \begin{cases} I_MAX & (I + P \times dt > I_MAX) \\ -I_MAX & (I + P \times dt < -I_MAX) \\ I + P \times dt & \text{otherwise} \end{cases}$$

PID Control: Derivative Term (D)

- D term differentiates the error between the current angle and target angle over time.
- Larger D_GAIN suppresses overshoot, but too large a value increases settling time.

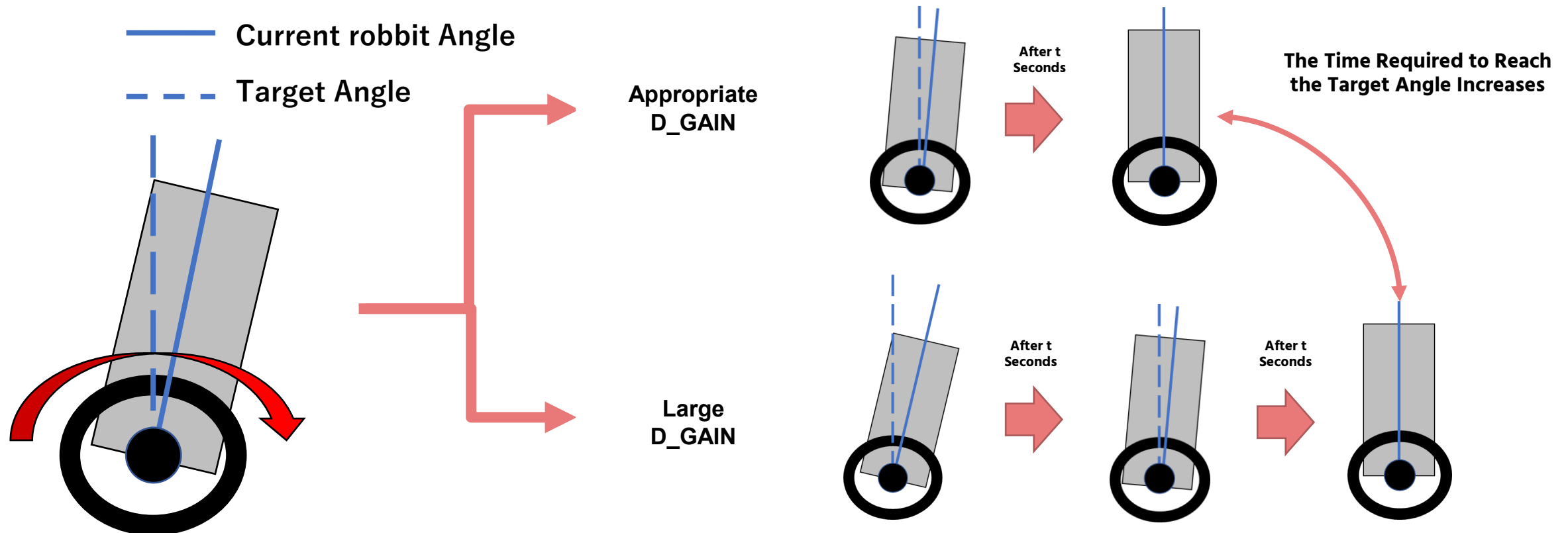


Illustration of the Derivative Term's Effect

Software: PID Control

The PID controller is implemented as follows.

```
// PID control
P = (parameter.target - roll) / 90.0;      Calculate Proportional Term
if(fabsf(I + P * dt) < I_MAX) I += P * dt; // cap Calculate Integral Term (Anti-Windup)
D = (P - preP) / dt;      Calculate Derivative Term
preP = P;

power = parameter.Kp * P + parameter.Ki * I + parameter.Kd * D; Calculate Output
```

- P is the error between the target angle (TARGET) and current angle (roll).
 - I integrates the proportional error using a rectangular approximation.
- The ``if (fabsf(I + P * dt))`` expression implements the anti-windup condition.
- D is calculated using the finite-difference derivative formula.
 - Finally, the terms are combined in power to calculate the output.

Software: Determining the PWM Signal

As the control signal for PWM control, the ratio of the waveform being HIGH is represented using PWM_base and V_MAX. る

The equation is shown below.

power represents the calculation result of PID control, and pwm_base represents the offset added to the PWM signal.

$$\text{PWM} = \begin{cases} \text{Power} + \text{PWM_base} & (|\text{power}| < \text{V_MAX}) \\ \text{V_MAX} & (|\text{power}| \geq \text{V_MAX}) \end{cases}$$

In addition, the sign of power is used as a control signal to determine the rotation direction of the motor.